PNAS RESEARCH ARTICLE | COMPUTER SCIENCES OPEN ACCESS

Tipping point forecasting in nonstationary dynamics on function spaces

Miguel Liu-Schiaffini[a], Clare E. Singer[b], Nikola Kovachki[c], Sze Chai Leung[a], Hyunji Jane Bae[a], Kamyar Azizzadenesheli[c], and Anima Anandkumar[a,1]

Edited by Igor Mezic, University of California, Santa Barbara, CA; received January 6, 2026; accepted September 7, 2026 by Editorial Board Member Alan Hastings

Tipping points are abrupt, drastic, and often irreversible changes in the evolution of nonstationary and chaotic dynamical systems. For instance, increased greenhouse gas concentrations are predicted to lead to drastic decreases in low cloud cover, referred to as a climatological tipping point. In this paper, we learn the evolution of such nonstationary dynamical systems using a recurrent neural operator (RNO), which learns mappings between function spaces. After training RNO on only the pre-tipping dynamics, we employ it to detect future tipping points using an uncertainty-based approach. In particular, we propose a conformal prediction framework to forecast tipping points by monitoring deviations from physics constraints (such as conserved quantities and partial differential equations), enabling forecasting of these abrupt changes along with a rigorous measure of uncertainty. We illustrate our proposed methodology on nonstationary ordinary and partial differential equations, such as the Lorenz-63 and Kuramoto–Sivashinsky equations. We also apply our methods to forecast a climate tipping point in stratocumulus cloud cover and airfoil wake and stall transitions using only limited knowledge of the governing equations. For the latter, we show that our proposed method zero-shot generalizes to forecasting multiple future tipping points under varying Reynolds numbers. In our experiments, we demonstrate that even partial or approximate physics constraints can be used to accurately forecast future tipping points.

AI | tipping points | neural operators | physics informed learning

Significance

Tipping points are abrupt and often irreversible shifts in complex systems, like the Earth's climate or fluid flows. We introduce a machine learning framework to forecast future tipping points. While most prior works predict tipping points in simpler systems, we demonstrate that our approach works on large-scale complex problems, including forecasting a climate tipping point in cloud cover and aerodynamic instability in airfoils.

Non-stationary chaotic dynamics are a prominent part of the world around us. In chaotic systems, small perturbations in the initial conditions significantly affect the long-term trajectory of the dynamics. Nonstationary chaotic systems possess further complexity due to their time-varying nature. For instance, the atmosphere and ocean dynamics that govern Earth's climate are modeled by highly nonlinear partial differential equations (PDEs). They exhibit nonstationary chaotic behavior due to changes in anthropogenic greenhouse gas emissions, insolation, and a myriad of complex internal feedbacks (1, 2) (Fig. 1*A*). Additionally, turbulent aerodynamics is governed by the Navier–Stokes equations, a set of nonlinear PDEs that describe the motion of fluids. These equations give rise to chaotic and nonstationary behavior across a wide range of scales, especially in high-Reynolds-number flows commonly encountered in aerospace applications (5). One of the main areas in scientific computing is understanding such phenomena and providing computational methods to model their dynamics. Numerical methods, e.g., finite element and finite difference methods, have been widely used to solve PDEs. However, numerical methods have enormous computational requirements to capture the fine scales in complex processes. Moreover, they do not provide a manageable way to learn from data to reduce scientific modeling errors.

1.1. Learning in Nonstationary Physical Systems. These complexities in modeling complex physical systems make learning the dynamics of their evolution in function spaces notoriously difficult. Prior works proposed various neural networks, such as recurrent neural networks (RNNs) (6) and reservoir computing (7), for learning such dynamical systems (6). However, these methods learn maps between finite-dimensional spaces and are thus not suitable for learning on function spaces, which are inherently infinite-dimensional objects. For example, when modeling spatiotemporal systems, the size of input and output layers in RNNs depend on the number of discretization points in physical space; the parameter count of RNNs depends on the input and output resolutions.

Author affiliations: [a]Division of Engineering and Applied Science, California Institute of Technology, Pasadena, CA 91125; [b]University of Colorado Boulder, Boulder, CO 80309; and [c]Nvidia Corporation, Santa Clara, CA 95051

Author contributions: M.L.S., N.K., K.A., and A.A. designed research; M.L.S. performed research; C.E.S. provided expertise on the cloud cover tipping point; S.C.L. and H.J.B. provided expertise on the airfoil setting; and M.L.S., C.E.S., S.C.L., H.J.B., K.A., and A.A. drafted the manuscript text.

The authors declare no competing interest.

This article is a PNAS Direct Submission. I.M. is a guest editor invited by the Editorial Board.

Copyright © 2026 the Author(s). Published by PNAS. This open access article is distributed under Creative Commons Attribution License 4.0 (CC BY).

[1]To whom correspondence may be addressed. Email: anima@caltech.edu.

This article contains supporting information online at https://www.pnas.org/lookup/suppl/doi:10.1073/pnas.2600331123/-/DCSupplemental.

Published October 15, 2026.

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

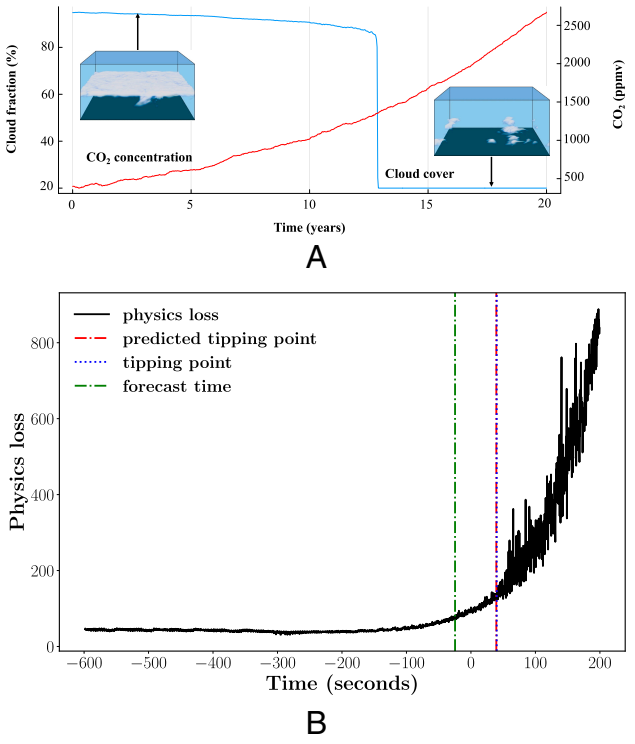


Fig. 1. (*A*) Tipping point in cloud fraction as a function of CO_2 concentration, from bulk model of the atmospheric boundary layer developed in refs. 3 and 4. 3d cloud cover renderings reproduced from ref. 2. (*B*) RNO accurately forecasts tipping point 64 seconds ahead in nonstationary Lorenz-63 system. "Predicted tipping point" is the time at which our framework predicts a tipping point will occur, and "forecast time" is the time when our framework makes this prediction.

Recent research introduced neural operators (8, 9) for learning operators to remedy these shortcomings. Neural operators receive functions at any discretization as inputs, possess multiple layers of nonlinear integral operators, and output functions that can be evaluated at any point. Neural operators are universal approximators of operators and can approximate continuous nonlinear operators in PDEs (8). Crucially, the number of parameters in neural operators stays constant with respect to the discretization of the inputs and outputs. These properties make neural operators discretization-invariant and suitable for learning in function spaces (10). Fourier neural operators (FNO) are architectures that use Fourier representations to integrate, providing an efficient implementation for many real-world applications. These methods, in particular Markov neural operators (MNO), have shown significant progress in learning Markov kernels of stationary dynamics systems (11). However, nonstationary dynamical systems have potentially long memories, making earlier MNO developments not directly applicable to the setting of this paper.

In this work, we introduce recurrent neural operators (RNOs), a recurrent architecture for neural operators that enables learning memory-equipped maps on function spaces (Fig. 3). RNO receives a sequence of historical continuous (in time) function data and represents the memory in terms of a latent state function. This enables conditional predictions for future functions given the latent functional representations of the past. In contrast to RNNs and other fixed-discretization approaches, RNO is discretization invariant in both space and time. We show that RNOs are capable of learning the dynamics of nonstationary chaotic systems and outperform RNNs and the state-of-the-art MNO by orders of magnitude on a model of stratocumulus cloud cover evolution (3, 4), as well the nonstationary Lorenz-63 system and the Kuramoto–Sivashinsky (KS) equation.

1.2. Tipping Point Forecasting. A commonly observed trait of nonstationarity is the existence of abrupt, drastic, and often irreversible changes in system dynamics, known as tipping points (12). Tipping points are conceptual reference points in nonstationary systems where the evolution undergoes a sudden change in quantities of importance. For instance, in the climate system, several tipping point phenomena have been identified (13–15), such as the collapse of the oceanic thermohaline circulation (16–18), ice sheet instability (19–22), permafrost loss (23, 24), and low cloud cover breakup (2). As shown in Fig. 1*A*, an increase in CO_2 in an idealized model is linked to a delayed drastic drop in cloud cover (2–4). Another example of tipping points are the wake transition and static aerodynamic stall in a flow over an airfoil with an increasing angle of attack (25, 26).

Predicting tipping points in nonstationary chaotic systems poses significant challenges due to several reasons. First, chaotic systems are highly sensitive to initial conditions, making precise long-term forecasting inherently uncertain. Small variations in initial states can lead to divergent outcomes, complicating the identification of critical thresholds where abrupt transitions occur. Second, the dynamics near tipping points often exhibit complex nonlinear behaviors that defy straightforward analytical or numerical predictions. This complexity arises from the interaction of multiple variables and feedback mechanisms, further complicating the identification of impending transitions. In the context of climate change and airfoil stall, where small changes can lead to large-scale effects such as extreme weather events or loss of lift, accurately pinpointing tipping points is crucial yet remains a formidable scientific and computational challenge.

Identifying and analyzing tipping points in chaotic systems often involves running numerical solvers over extended time horizons, which results in significant computational expense (2, 17). Therefore, solvers are often run at low resolution or over subsets of the full domain, which can result in loss of accuracy or generality (2).

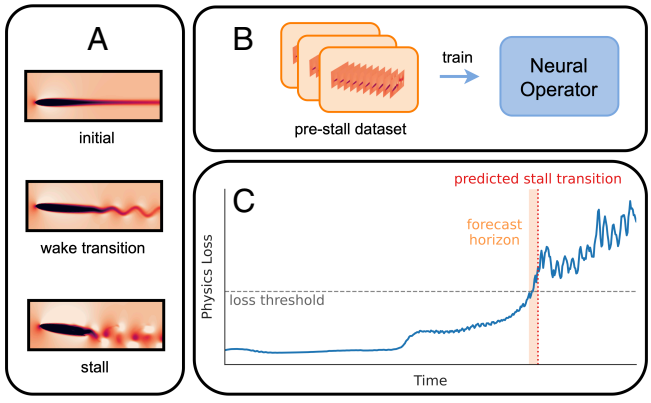


Fig. 2. Conceptual diagram of our proposed method applied to a flow over an airfoil with a changing angle of attack. (*A*) The system exhibits two tipping points, the wake and stall transitions. In this example, we consider forecasting the stall transition. (*B*) We train a neural operator model to learn the pre-stall dynamics of the system. Note that our method does not require access to post-tipping data at training time. (*C*) At inference time, we autoregressively roll-out our model's dynamics predictions and compute the predictions' physics loss at each time-step. The first time the physics loss crosses a predefined loss threshold, we label this as the tipping point.

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

Recently, machine learning methods have been deployed for predicting the tipping points of nonstationary systems (7, 27–37). However, prior works either focus on detecting domain shift instead of forecasting tipping points far ahead in time, and those that do forecast tipping points lack a systematic approach to learn nonstationary systems at arbitrary resolutions and forecast tipping points in function space.

In this work, we propose training an RNO model to predict future states on the complex dynamics of nonstationary chaotic systems using only data of pre-tipping point events. The set-up is motivated by climate modeling, where tipping points are not present in forty years of historical data (38). To forecast the time of a tipping point, we build our approach based on the common observation that, at a tipping point, the prediction accuracy of machine learning models degrades due to distribution shifts (7, 28). Since in our setting we do not have access to post-tipping data, we instead propose to forecast tipping points whenever the model prediction of the future exhibits extensive violations of domain-specific physical constraints. We use these physical constraints (e.g., conservation laws or governing PDEs) to verify the correctness of our model's system forecasts at a given future time. Our method assumes that a given RNO model is well trained on pre-tipping dynamics. Crucially, since the model is trained only on pre-tipping dynamics, even a perfect model on the pre-tipping regime would degrade in prediction accuracy beyond the tipping point, as observed empirically. We show that this approach predicts tipping points very far in advance. See Fig. 2 for a conceptual diagram.

To quantify the prediction accuracy, we propose a conformal prediction method based on cumulative distribution functions (CDFs) (39). We estimate the CDF of physics error, which quantifies the violation of physics constraints by the model's forecasts and plays the role of a nonconformity score. We employ the fact that using finitely many samples, a CDF of any distribution can be estimated accurately everywhere under the Kolmogorov–Smirnov distance, a property that is not true for other functions like quantiles (40, 41). Using the empirical CDF of the physics error, we quantify the distribution of the nonconformity score. This uncertainty quantification is directly connected to the false positive rate of our prediction approach.

We consider a real-world system describing tipping points in stratocumulus cloud cover (3, 4) and wake and stall tipping points in flow over an airfoil (26). We also consider the more classical nonstationary and chaotic Lorenz-63 and Kuramoto–Sivashinsky (KS) systems. We show that RNO consistently outperforms the state-of-the-art adapted baseline MNO in forecasting tipping points in these systems, particularly for the KS equation. Furthermore, we demonstrate under our proposed approach, RNO is able to accurately forecast tipping points when run for up to 200 time intervals (Fig. 1*B*). Our proposed method can also forecast real-world tipping points, such as the aforementioned cloud cover transition, using only partial knowledge of the underlying physics (Fig. 5*A*). It does so with an error of only 10 d while forecasting nearly 2.5 years in advance.

To demonstrate the scalability of our method to high-dimensional settings, we consider the problem of forecasting the wake and stall transitions in a flow over an airfoil. We find that we are able to successfully forecast the tipping points using limited knowledge of the underlying governing equations (specifically, using only the divergence-free condition) and for various airfoil pitching rates. Further, our method is capable of forecasting the stall tipping point while trained only on pre-wake data, as well as generalizing to forecasting tipping points at a Reynolds number of 5000 while only being trained on Reynolds number 1000 trajectories. We find that our method is also capable of avoiding false positive forecasts when the underlying nonstationary system does not exhibit a tipping point.

In summary, we propose a tipping point forecasting framework that scales to arbitrary tipping points and arbitrary spatiotemporal systems on function spaces. In particular, we

1. Introduce RNO, an operator learning architecture to learn complex nonstationary dynamics, which far outperforms Markovian and RNN baselines in our experiments. For instance, RNO provides up to 70.9% improvement in 20-step autoregressive prediction on the nonstationary Kuramoto–Sivashinsky equation.
2. Construct a tipping point forecasting framework that relies only on 1) a data-driven model of pre-tipping dynamics and 2) a physical constraint of the system.
3. Propose a conformal prediction method to quantify deviations in model physics error in a statistically rigorous manner.
4. Achieve accurate tipping point forecasting far in advance and demonstrate our method's scalability to spatiotemporal PDEs such as wake and stall transitions in a flow over an airfoil.
5. Show high accuracy in tipping point forecasting using approximate and partial physical constraints, generalizing our method to settings when full physics laws are unknown.
6. Demonstrate high accuracy in zero-shot tipping point forecasting in out-of-domain regimes unseen at training time.

2. Related Work

Tipping points exhibit complex behavior, making their study a challenging task in the high-dimensional multimodal setting (1). In recent years, tipping points and their root causes have been categorized for simple systems (such as low-dimensional ODEs), based on stochasticity, rate change, and bifurcations (12, 42, 43). Tipping points can be caused by changes in the condition and sourcing in differential equations, or sometimes, the root cause is the rate at which these conditions change (12, 44). However, in the case of general scientific computing problems on natural phenomena, theoretical studies of tipping points are very challenging, and prior works have primarily focused on their empirical evaluation via numerical simulation.

Tipping points are abundant in scientific computing of climate evolution and control modeling (13–15). Previous work has highlighted the importance of predicting tipping points in the Earth system with the goal of giving early warning ahead of large changes in the climate (45, 46). Approaches have varied depending on the type of tipping behavior considered (47). For instance, it has been shown that changes in atmospheric CO_2 levels can cause rapid changes in low cloud cover on the Earth (2, 4). And furthermore, after bringing CO_2 levels below the tipping threshold, these changes may not be readily undone (the system exhibits hysteresis).

Traditional early warning signals (EWS) of tipping points such as increased autocorrelation before tipping events have been studied extensively for many simple systems (46, 48) and for some specific real-world data (49). In the stationary regime, these empirical signals rest on a rigorous mathematical foundation: The statistical properties of systems near critical transitions are characterized through the spectrum (particularly by the spectral gap) of the generator of the transfer semigroup (50–52). Finally, tipping points in the large spatiotemporal systems that motivate

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

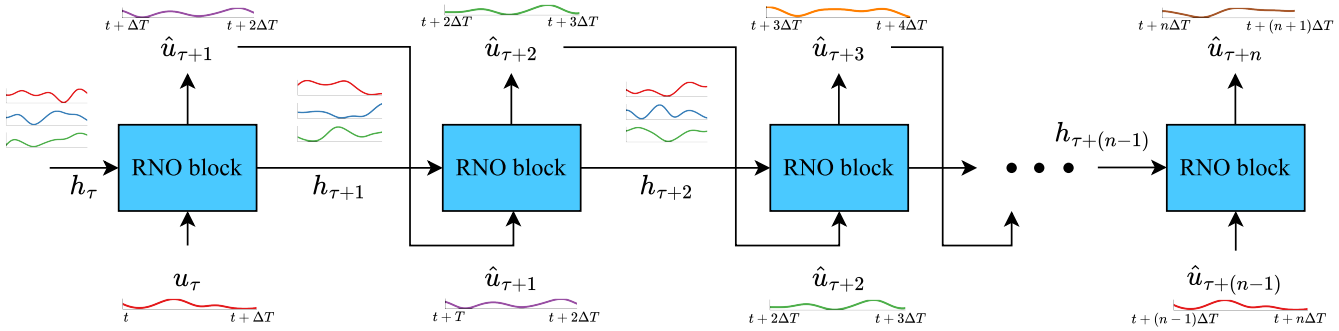


Fig. 3. Illustration of RNO autoregressive forecasting.

this work have not yet been sufficiently studied empirically and theoretically for these indicators to become evident, and for many spatiotemporal systems, thorough empirical evaluation is difficult due to the computational cost of simulations. Thus, data-driven approaches for learning system dynamics and forecasting tipping points is critical for most challenging real-world problems. In *SI Appendix*, section S9.B, we demonstrate that traditional EWS are not reliable at predicting tipping points in the systems we study. Recently, machine learning methods have been deployed for learning in nonstationary dynamical systems and to predict their tipping points (7, 27–37). However, many works require post-tipping data (7, 27), large datasets of labeled time series (34), full governing equations (30), or are not generalizable to arbitrary types of tipping points (27, 32, 36) or large spatiotemporal systems (35). In contrast, our method is scalable, does not need full governing equations, and is not trained on post-tipping data. See Table 1 and *SI Appendix*, section S9.A for a detailed comparison with prior work.

RNNs are often used to learn in finite-dimensional dynamical systems with memory or in time-series with discrete-time dynamics (6, 53). They consist of blocks of neural networks that carry memory as a finite-dimensional latent state and enable conditional predictions on learned latent historical representations. But since neural networks are maps between finite-dimensional spaces, RNNs are unfit for many scientific computing problems, which often involve *functions* (i.e., infinite-dimensional objects) and their time-evolution. Neural ODE versions of RNNs have similar issues, can only model specific time derivatives, and require expensive solvers for temporal simulation (54–56).

Neural operators are deep learning models that generalize neural networks to maps between function spaces (9, 57, 58), and they are universal approximators of general operators (8). The inputs to neural operators are functions, and the output function can be evaluated at any point in the function domain. These models are thus known to be *discretization invariant* and can take the input function at any resolution. In this paper, we develop the recurrent neural operator (RNO). RNO receives a sequence of historical function data and represents the memory in terms of a latent state function, enabling conditional predictions given learned latent functional representations of the past.

After the initial version of this paper (59) was released, a few related methods were introduced in the literature. Ref. 60 introduce MemNO, an architecture that combines state space models with FNO by introducing memory, and ref. 61 propose a memory-free architecture where the model inputs are short time intervals. Ref. 62 train neural operators autoregressively for better long-time rollouts. Ref. 63 factorize the spatial and temporal dimensions and propose to use structured state space models to model each dimension.

Table 1. Comparison of methods

Method	General	Func. space	Physics	Speed	Pre-tip
Solver	✓	✓	×	×	N/A
EWS (45)	×	×	N/A	✓	✓
Bury et al. (27)	×	×	N/A	✓	×
Patel and Ott (7)	✓	×	N/A	✓	×
Ours	✓	✓	✓	✓	✓

"Generality" denotes applicability to arbitrary types of tipping points. "Pre-tip data" denotes whether the method can forecast tipping points using only pre-tipping data.

3. Materials and Methods

3.1. Preliminaries. A PDE can be seen as a differential law on function spaces. For an input function a from a function space $\mathcal{A}$ (e.g., a fluid velocity field), we denote $\mathcal{D}_\mathcal{A}$ as the domain and $\mathbb{R}^{d_\mathcal{A}}$ as the function codomain with dimension $d_\mathcal{A}$. For any point in the domain $x \in \mathcal{D}_\mathcal{A}$, the function a maps this point to a $d_\mathcal{A}$-dimensional vector, i.e, $a(x) \in \mathbb{R}^{d_\mathcal{A}}$ (e.g., the velocity of a fluid at a particular spatial location x). Correspondingly, let $\mathcal{U}$ denote the solution or output function space (e.g., the fluid velocity field at a later time), defined on domain $\mathcal{D}_\mathcal{U}$ with codomain $\mathbb{R}^{d_\mathcal{U}}$, i.e., for any function $u \in \mathcal{U}$ and any $x \in \mathcal{D}_\mathcal{U}$, we have $u(x) \in \mathbb{R}^{d_\mathcal{U}}$. For a given a, we define a PDE in generic form as,

$$\begin{aligned} \mathcal{L}(u,a)(x) &= 0, \qquad \text{in } \mathcal{D}_\mathcal{U}, \\ \mathcal{L}'(u,a)(x) &= 0, \qquad \text{in } \partial\mathcal{D}_\mathcal{U}, \end{aligned} \qquad [1]$$

where $\mathcal{L}(u,a)$ is the governing law in the domain $\mathcal{D}_\mathcal{U}$, and $\mathcal{L}'(u,a)(x)$ is the constraint on the domain boundary $\partial\mathcal{D}_\mathcal{U}$. Following the velocity field example, this may be the Navier–Stokes equations with a given set of boundary conditions. A function u is a solution to the above PDE at input a if the function u satisfies both of the PDE constraints (Eq. **1**), comprising an input-solution pair (a,u). We are concerned with learning maps from input function spaces $\mathcal{A}$ to output function spaces $\mathcal{U}$ in operator learning (e.g., learning the map of a fluid velocity field from time t to $t+h$). For a given PDE, let $\mathcal{G}^\star$ denote the operator that maps the input functions in $\mathcal{A}$ to their corresponding solution functions in $\mathcal{U}$, which we seek to learn with a neural operator.

A neural operator $\mathcal{G}$ is a deep learning architecture consisting of multiple layers of point-wise and integral operators (8) to learn a map from input function a to output function u. The first layer of the architecture is a pointwise lifting operator $\mathcal{P}_0$ such that for a given function a, the output of this layer $v_0 = \mathcal{P}_0 a$ is computed so that for any $x \in \mathcal{D}_\mathcal{A}$, $v_0(x) = P_0(a(x)) \in \mathbb{R}^{d_0}$ where P_0 is a learnable neural network. This follows standard deep learning architectural practice, where the computation is performed in a higher-dimensional latent space. This step is followed by L layers of nonlinear integral operators. For any layer l we have,

$$v_l = \mathcal{P}_l(\sigma(\mathcal{K}_l v_{l-1}) + \mathcal{W}_l v_{l-1} s_l), \qquad [2]$$

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

where $v_l : D_l \to \mathbb{R}^d_l$, σ is some pointwise nonlinearity, and $\mathcal{K}_l$ is an integral operator such that for any $x \in \mathcal{D}_l$, we have,

$$\mathcal{K}_l v_{l-1}(x) = \int_{\mathcal{D}_{l-1}} \kappa_l(x,y) v_{l-1} d\mu_l(y) + \mathcal{W}'_l v_{l-1}(s_l(x)). \quad [3]$$

Here κ_l is a learnable kernel function, $\mathcal{W}'_l$ is a pointwise operator parameterized by a neural network W'_l, and s_l is a deformation map from $\mathcal{D}_l$ to $\mathcal{D}_{l-1}$. Similarly, $\mathcal{W}_l$ is a pointwise operator parameterized by W_l neural network, and $\mathcal{P}_l$ is a pointwise operator parameterized by a neural network P_l. μ_l represents the measure on each space and finally, we set $\mathcal{D}_L = \mathcal{D}_{\mathcal{U}}$. Intuitively, this kernel integral generalizes matrix multiplication in standard neural networks by integrating over the spatial domain $\mathcal{D}_{\mathcal{U}}$, where the specific discretization points enter as quadrature in the integral (10). Parameterizing the kernel κ_l with a finite number of parameters makes the parameter count of a neural network fixed with respect to the spatial discretization. The last layer is a pointwise projection operator $\mathcal{Q}$ parameterized by a neural network Q. In particular, we use Fourier representations to compute the integration Eq. **3** in its convolution form,

$$\mathcal{K}_l v_{l-1}(x) = \mathcal{F}^{-1}\left(F\kappa_l \cdot \mathcal{F} v_{l-1}\right) + \mathcal{W}'_l v_{l-1}(s_l(x)), \quad [4]$$

where $\mathcal{F}$ denotes the Fourier transform and projects its inputs to k_l Fourier bases, for some hyperparameter k_l. This represents the integral operator with a finite number of parameters but still exhibits invariance to the input and output resolutions. Let R_l denote the k_l Fourier coefficients of κ_l, i.e., $R_l = \mathcal{F}\kappa_l$. Following ref. 57, we directly parameterize R_l instead of κ_l to learn the integral operator. We utilize these building blocks to construct the RNO architecture in the next section.

3.2. Recurrent Neural Operator (RNO). We now describe and formulate RNO, a generalization of RNNs to function spaces. For a given PDE describing the evolution of a function $u(x,t)$ of time and space, consider a partition of the time domain t into equally spaced intervals of length ΔT and indexed by $\tau \in \mathbb{N}$. A step τ is associated with the interval $[(\tau-1)\cdot\Delta T, \tau\cdot\Delta T]$, where the input function restricted to this interval is defined for $t \in [t, t+\tau\cdot\Delta T]$ as $u_\tau(x,t) := u(x,t)$.

Our proposed architecture of the RNO parallels that of the gated recurrent unit (GRU) (64), where the GRU cell at step τ receives a hidden state vector h_τ and an input vector x_τ, and predicts the next output vector $x_{\tau+1}$ and next hidden state vector $h_{\tau+1}$, which is also passed to the GRU cell at step $\tau+1$. The key difference between the RNO and the GRU is that at step τ, an RNO cell receives as input a hidden function $h_\tau(x,t)$ (which itself is represented at arbitrary resolutions) and an input function $u_\tau(x,t)$. As such, we replace the linear maps in the GRU architecture with Fourier layers, Eq. **4**). Given u_τ and h_τ, we define the reset gate (which intuitively determines what to forget from the past) such that for any $t \in [t, t+\tau\cdot\Delta T]$,

$$r_\tau(x,t) = \sigma\Big((\mathcal{K}_r u_\tau)(x,t) + (\mathcal{K}'_r h_\tau)(x,t) + b_r(x,t)\Big), \quad [5]$$

where $\mathcal{K}_r, \mathcal{K}'_r$ are Fourier layers, $b_r(x,t)$ is a learned bias function, and σ is the sigmoid function applied pointwise. We define the update gate (which intuitively determines what to remember for the future) as,

$$z_\tau(x,t) = \sigma\Big((\mathcal{K}_z u_\tau)(x,t) + (\mathcal{K}'_z h_\tau)(x,t) + b_z(x,t)\Big). \quad [6]$$

We define the candidate hidden function as,

$$\widehat{h}_{\tau+1}(x,t) = \phi\Big((\mathcal{K}_h u_\tau)(x,t) + (\mathcal{K}'_h(r_\tau \cdot h_\tau))(x,t) + b_h(x,t)\Big), \quad [7]$$

where ϕ denotes a point-wise scaled exponential linear unit (SELU) activation and the function multiplication $(\cdot)$ is taken to be pointwise. Finally, we define the hidden state function $h_{\tau+1}(x,t)$ as

$$h_{\tau+1}(x,t) = \Big((1-z_\tau)\cdot h_\tau\Big)(x,t) + \Big(z_\tau \cdot \widehat{h}_{\tau+1}\Big)(x,t). \quad [8]$$

We take $h_0(x,t)$ to be a learned prior function on the initial hidden state. Let $\mathrm{RNO}(v, h_\tau)$ denote an RNO cell. Given the input function u_τ, an RNO block takes the following form:

$$\tilde{u}_\tau(x,t) = \mathcal{P}_0(u_\tau)(x,t) = P_0(u_\tau(x,t)), \quad [9]$$

$$v_\tau^{(i+1)}(x,t) = \mathrm{RNO}^{(i+1)}\left(v_\tau^{(i)}, h_\tau^{(i+1)}\right)(x,t), \quad [10]$$

$$\widehat{u}_\tau(x,t) = \mathcal{Q}(v_\tau^{(L)})(x,t) = Q(v_\tau^{(L)}(x,t)), \quad [11]$$

where $\mathcal{P}_0$ and Q are pointwise lifting and projection operators, respectively parameterized by neural networks P_0 and Q. $v_\tau^{(0)}(x,t) := \tilde{u}_\tau(x,t)$, the result of the lifting operation, and L is the number of RNO layers. An RNO is then a RNO block evolving in time with $h_{\tau+1}^{(i)} := v_\tau^{(i)}$ for each RNO cell $i \geq 1$, see Fig. 3.

We divide the inference stage of RNO into two phases:

1. **Warm-up phase:** In the warm-up phase, we have access to the ground-truth trajectory between steps $\tau = 0$ and $\tau = N_\tau - 1$, corresponding to the ground-truth solution $u(t)$ between times $t \in [0, N_\tau \Delta T]$. During this phase, the RNO blocks receive as input u_τ, and the predicted $\widehat{u}_{\tau+1}$ is discarded.
2. **Prediction phase:** In the prediction phase, we assume we no longer have access to the ground-truth trajectory beyond time $\tau = N_\tau - 1$, so the model predictions $\widehat{u}_{\tau+j}$ are taken as input to the RNO block at time $\tau + j + 1$, and the model is thus composed with itself (with access to its hidden state h_τ), with its outputs fed back on itself as inputs.

The prediction phase is similar to Markov neural operator (MNO) (11), with the exception that RNO now has access to a continually updating hidden representation $h_\tau(x,t)$, which encodes the history of the nonstationary system, making RNO a generalization of MNO to systems with memory.

3.3. Tipping Point Forecasting. Forecasting tipping points requires rigorous uncertainty quantification, and any prediction must be accompanied by the event's potential probabilities. Conformal prediction is a field of study that accompanies predictions with certainty levels in terms of a conformity score (39). Most prior work on traditional conformal prediction quantifies the probability distribution of the conformity score at certain levels. Another approach is to utilize quantile regression to quantify the distribution of conformity scores (65). Both of these approaches have shortcomings in tipping point prediction. The former approach utilizes exchangeability and requires new and fresh draws of samples to quantify the distribution of conformity scores, imposing a union bound over a long period of time. Quantile regression suffers from two downsides: 1) quantiles are not generally estimable, and 2) a union bound over an infinite set is required to quantify the conformity score distribution (66). These limitations are addressed below by introducing CDF estimation methods.

Our tipping point forecasting utilizes training on the pre-tipping dynamics of the system. We train RNO on datasets of time-evolving functions that do not contain any tipping points. We make this choice to resemble real-world settings such as climate where recorded historical data does not contain tipping points. At any time t, we use RNO to predict the long evolution of the dynamics for an interval of $[t, t+T]$, i.e., $\widehat{u}(x,[t,t+T])$ where T is a multiple of ΔT. At the forecast time t, we do *not* have access to the future $u(x,[t,t+T])$. If we did, we could use the deviation of $\widehat{u}(x,[t,t+T])$ from $u(x,[t,t+T])$ as a hindsight indicator for the tipping point as observed in prior works (7, 28).

Instead, we propose to use the physical constraints $\mathcal{L}(u) = 0$ of the underlying system (such as the PDE that governs the dynamics) to evaluate the deviation of $\widehat{u}(x,[t,t+T])$ from physics laws, and we use this signal as an indicator of tipping points. For a predicted function $\widehat{u}(x,[t,t+T])$, we define the conformity score as $\|\mathcal{L}(\widehat{u}) - 0\|$, and we refer to it by the physics loss $\ell_P(\widehat{u})$. Note that if $\widehat{u}(x,[t,t+T]) = u(x,[t,t+T])$ (the true future function), then $\mathcal{L}(\widehat{u}) = 0$, and thus our proposed indicator aligns with the hindsight indicator mentioned above. At any time t, we compute $\ell_P(\widehat{u})$, and if it takes values above a certain threshold, we forecast that a tipping point is anticipated. This method enables tipping point forecasting. However, since the trained model comes with training generalization error, this approach makes mistakes, confusing large errors on pre-tipping regimes and actual tipping incidence. This is of particular

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

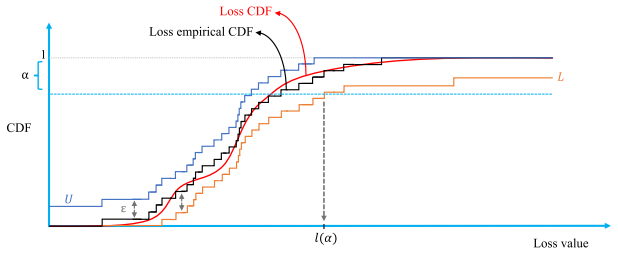


Fig. 4. Diagram of the setup for the tipping point prediction method. U and L correspond to the upper and lower bounds given by the CDF concentration inequality, differing from the empirical CDF by ε. α is the significance level, and $l(\alpha)$ is the corresponding loss of L at α.

importance when the forecasting interval of interest T is large. We propose the following conformal prediction to address uncertainty quantification in tipping point forecasting. For a trained model and given a forecasting interval T, let F denote the true CDF of the conformity score. The CDF captures the probabilities at which the model makes errors of a given magnitude on pre-tipping data. We use the CDF to develop our conformal prediction method, which allows for uncertainty quantification of tipping point forecasts. In particular, for any loss threshold at which we call a tipping point, we can denote α to be the probability of falsely forecasting a tipping point when the data was simply from the pre-tipping regime. We first compute the empirical CDF $\widehat{F}$ of the conformity score $\ell_P(u)$ using n calibration samples. Formally, given n samples, we construct $\{\ell_P^{(j)}\}_{j=1}^n$ and compute $\widehat{F}$ as follows. For any $p \in \mathbb{R}$,

$$\widehat{F}(p) := \frac{1}{n}\sum_{j}^{n} \mathbb{1}(p \geq \ell_P^{(j)}).$$

We then use the concentration inequality of CDFs to compute its upper and lower bounds (40, 41). Using the concentration inequality of CDFs, with probability at least $1-\delta$ we have,

$$\sup_{p\in\mathbb{R}} \left|\widehat{F}(p) - F(p)\right| \leq \varepsilon := \sqrt{\frac{\log(2/\delta)}{2n}},$$

where the left-hand side is the Kolmogorov–Smirnov distance between the two CDFs. We define the CDF lower bound $L(p) := \max\{0, \widehat{F}(p) - \varepsilon\}$ and the CDF upper bound $U(p) := \min\{1, \widehat{F}(p) + \varepsilon\}$. Fig. 4 depicts this procedure. Intuitively, α denotes the maximum probability of a mistake that one is willing to tolerate. In our approach, as shown in Fig. 4, we first find $l(\alpha)$, the conformity level at which the CDF lower bound L has probability $1-\alpha$. Using the level $l(\alpha)$, we call the presence of a tipping point whenever the conformity score of our prediction $\ell_P(\widehat{u})$ is above $l(\alpha)$.

Proposition 1. *For any given $\alpha \in [0,1]$, the decision rule of calling for a tipping point at level $l(\alpha)$ has a false positive rate of at most α, and this statement holds with probability at least $1-\delta$.*

The proof is based on the fact that for any α, $F^{-1}(1-\alpha) \leq l(\alpha)$ and $F(l(\alpha)) \geq 1-\alpha$. Therefore, when $\ell_P \geq l(\alpha)$, signaling a tipping point, we also have $\ell_P \geq F^{-1}(1-\alpha)$. Thus, the probability of the event $\ell_P \geq l(\alpha)$ is lower bounded by the probability of $\ell_P \geq F^{-1}(1-\alpha)$ which at most α. This proposition simultaneously holds for all the time steps τ and all α. This property enables assessing tipping predicting at various levels, a crucial feature for risk assessment and policy making.

4. Results

We present experimental results on learning dynamics and forecasting tipping points in nonstationary systems. We showcase the performance of RNO and the tipping point forecasting method on a simplified model of cloud cover equations and on predicting the turbulent wake and stall tipping points in a 2-d flow over an airfoil. In *SI Appendix*, section S1, we also present detailed results for the finite-dimensional Lorenz-63 system, and for the (infinite-dimensional) KS PDE in *SI Appendix*, section S2. For Lorenz-63, we show that our method is capable of forecasting tipping points far in advance *using only approximate knowledge of the underlying system*. In *SI Appendix*, section S7, we show that our method is robust to different rates of parameter change and can avoid predicting a tipping point when the underlying system lacks one. For the nonstationary KS equation, we show that RNO far outperforms MNO and RNN in state prediction for rollouts of various lengths. Discussion on the technical details of RNO (e.g., memory usage, inference times, hyperparameter selection, etc.) can be found in *SI Appendix*, section S10.

4.1. Learning Nonstationary Dynamics. We compare RNO against Markov neural operators (MNO) (11) and RNNs in forecasting nonstationary dynamics. There are two key properties that any learned model must exhibit: 1) low step-wise error and 2) error stability in long-time predictions (i.e., when the model is composed with itself many times). Low step-wise error is a necessary prerequisite to adequately learning the dynamics of any system. However, for many downstream tasks of significant scientific importance, it is also crucial that our model's error remain low when forecasting far into the future. This ensures the accuracy of tipping point prediction is credible in complex real-world settings. Hereafter, we define "n-step prediction" to be a model's forecasting prediction looking ahead $\tau = n$ steps.

In our experiments, we employ a multistep training regimen for all models, where the total loss is a combination of the loss on the model's n-step prediction for $n \in \{1, 2, \ldots, M\}$. That is, our data loss function is given by

$$\ell_D(u, \widehat{u}) = \sum_{n=1}^{M} \lambda_n \|u_n - \widehat{u}_n\|_2, \qquad [12]$$

where $\|\cdot\|_2$ is the L^2 norm and λ_n is some scaling hyperparameter.

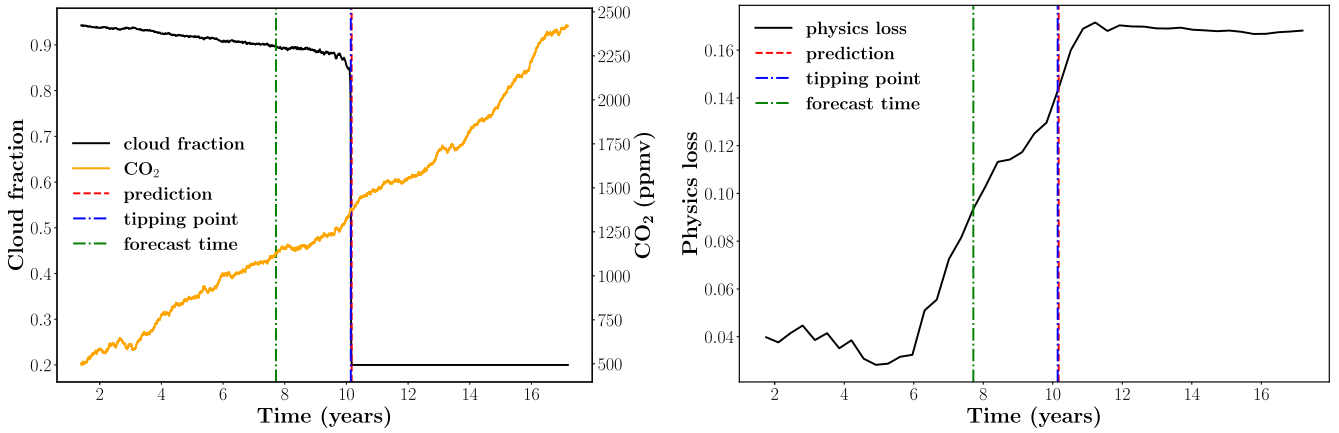
Table 2 compares the relative L^2 errors of RNO, MNO, and RNN on forecasting the nonstationary KS equation (setup and details in *SI Appendix*, section S2). We observe that RNO outperforms MNO and RNN on every forecasting interval. Further, we can observe the disadvantages of RNN and other finite-dimensional, fixed-resolution models on learning in PDE (i.e., infinite-dimensional) settings. The RNN's error is orders of magnitude larger than that of RNO or MNO for small forecasting windows. At large forecasting windows (i.e., 12-step and above), the lack of explicit history in MNO becomes a severe disadvantage. In contrast, RNO maintains much lower error at large forecasting intervals than the other two models. Both the MNO and RNO shown in Table 2 were trained with $M = 5$ steps of fine-tuning (Eq. **12**). Observe that MNO and RNN have difficulty generalizing to forecasting intervals larger than M, whereas RNO is capable of maintaining stable error even at longer forecasting intervals.

Table 2. Relative L^2 errors on the nonstationary KS equation for different n-step prediction settings

Model	1-step	2-step	4-step	8-step	12-step	16-step	20-step
RNO	0.0030	0.0027	0.0031	0.0419	0.1410	0.2919	0.4008
MNO	0.0050	0.0073	0.0150	0.2901	1.0751	1.2626	1.3772
RNN	0.0525	0.0481	0.1443	0.5488	0.9031	1.0031	1.1561

RNO outperforms both MNO and the baseline RNN on every forecasting intervals.

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.



A Tipping point forecast w.r.t. cloud fraction

B Tipping point forecast w.r.t. partial physics loss

Fig. 5. For the cloud cover model, (*A*) appropriately stylized, (*B*) shows the tipping point forecasting time as a function of the physics loss, using only a mass conservation constraint (as opposed to the full system), RNO is still capable of identifying the true tipping point of the system with an error of 0.03 years, predicting $T = 2.45$ years ahead, at $\alpha = 0.07$.

4.2. Cloud Cover Equations: Tipping Points from Partial Physics. We now consider a case study of tipping point forecasting in the Earth's climate. We also demonstrate that our method is still accurate under partial physics, e.g., conservation laws (not the full governing equations). We use the ODE model developed in refs. 3 and 4 as an idealization of the physical climate system; this model exhibits a tipping point. The model represents a stratocumulus-topped atmospheric boundary layer, which experiences rapid loss of cloud cover under high CO_2 concentrations and shows hysteresis behavior, where the original state is not immediately recovered when CO_2 concentrations are lowered. This model was developed, in part, to help explain the results found from the high-resolution simulations in refs. 2 and 67. More details on the model can be found in *SI Appendix*, section S3, and details on numerical experiments can be found in *SI Appendix*, section S8.D.

We train an RNO to forecast the nonstationary evolution of the system with a data loss given by Eq. **12**, which incorporates the L^2 loss across longer time horizons. We compute the empirical CDF of the physics loss for the cloud cover system (analogous to *SI Appendix*, Eq. **S5** in the KS setting) as a function of model predictions and CO_2. However, in this case, the physics loss is normalized component-wise by the state of the system at a given time, since the magnitudes of each component in this model differ by several orders.

For tipping point forecasting, we base the physics loss solely on mass conservation (*SI Appendix*, Eq. **S6a**), instead of the full ODE. Specifically, we let ℓ_P be the L^2 norm of the difference between the left and right-hand-sides of Eq. S6a, which are both functions of time within a time interval. In our experiments, we set the tipping point forecasting horizon to be $T = 7 \cdot \Delta T = 2.45$ years (with forecasting intervals of $\Delta T = 0.36$, or 128 d). We found that the choice of ΔT (on time-scales on the order of $O(100)$ days) does not significantly impact the RNO's ability to learn the dynamics. T is chosen to be an order of magnitude larger than ΔT, to demonstrate that our framework is capable of forecasting tipping points at time-scales much longer than ΔT. The results on a given test trajectory can be seen in Fig. 5. Despite the abruptness of the tipping point and the highly nonlinear dynamics of the system, our forecasting framework is successfully able to identify the true tipping point in this system using a physics loss derived from a partial picture of the full physics.

4.3. Forecasting Airfoil Wake and Stall Transitions. To demonstrate the scalability of our method to complex higher-dimensional settings, we consider the problem of forecasting two tipping points in a 2-d flow over an airfoil with angle of attack that increases at a constant rate over time. We simulated the flow past an airfoil of chord length C with velocity U_∞ by solving the 2-d incompressible Navier–Stokes equation using a large-eddy simulation (LES) solver (more details can be found in *SI Appendix*, section S4 and in ref. 68). Two distinct tipping points in the lift coefficients can be observed as a function of the angle of attack (alternatively, as a function of time, since the angle of attack increases at a constant rate over time). The first tipping point observable in the lift coefficient is known as the critical bifurcation from singular-vortex shedding to vortex-pair shedding, which we refer to simply as the wake transition (25). The second tipping point corresponds to the static aerodynamic stall, which is characterized by a rapid drop in the lift coefficient for a small increase in the angle of attack (26). At low Reynolds numbers (Re), however, this drop tends to occur more gradually at angles of attack around 15 degrees (69).

In our experiments, we consider three settings: 1) a Re of 1000, where only the static stall occurs, 2) the wake transition in a flow with Re 5000, and 3) static stall in a flow with Re 5000. Creating these three setups allows us to create a direct progression in the difficulty of each subtask. For instance, we would expect a priori that forecasting 1) is easier than forecasting 2) or 3) because the Re 1000 dynamics are less turbulent than the Re 5000 dynamics. Similarly, we would a priori expect that 3) is more challenging than 2) because it requires forecasting one tipping point after another and thus would require a model that has learned dynamics under multiple regimes.

We first train three separate RNO models to forecast the nonstationary pre-tipping evolution of systems 1), 2), and 3) using the data loss Eq. **12**. The tipping point forecasting methodology is identical to Section 4.2, except the physics loss is now defined by the divergence-free condition of incompressible Navier–Stokes $\ell_P(u) = \|\nabla \cdot u\|_2$, where u is the velocity field. We

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

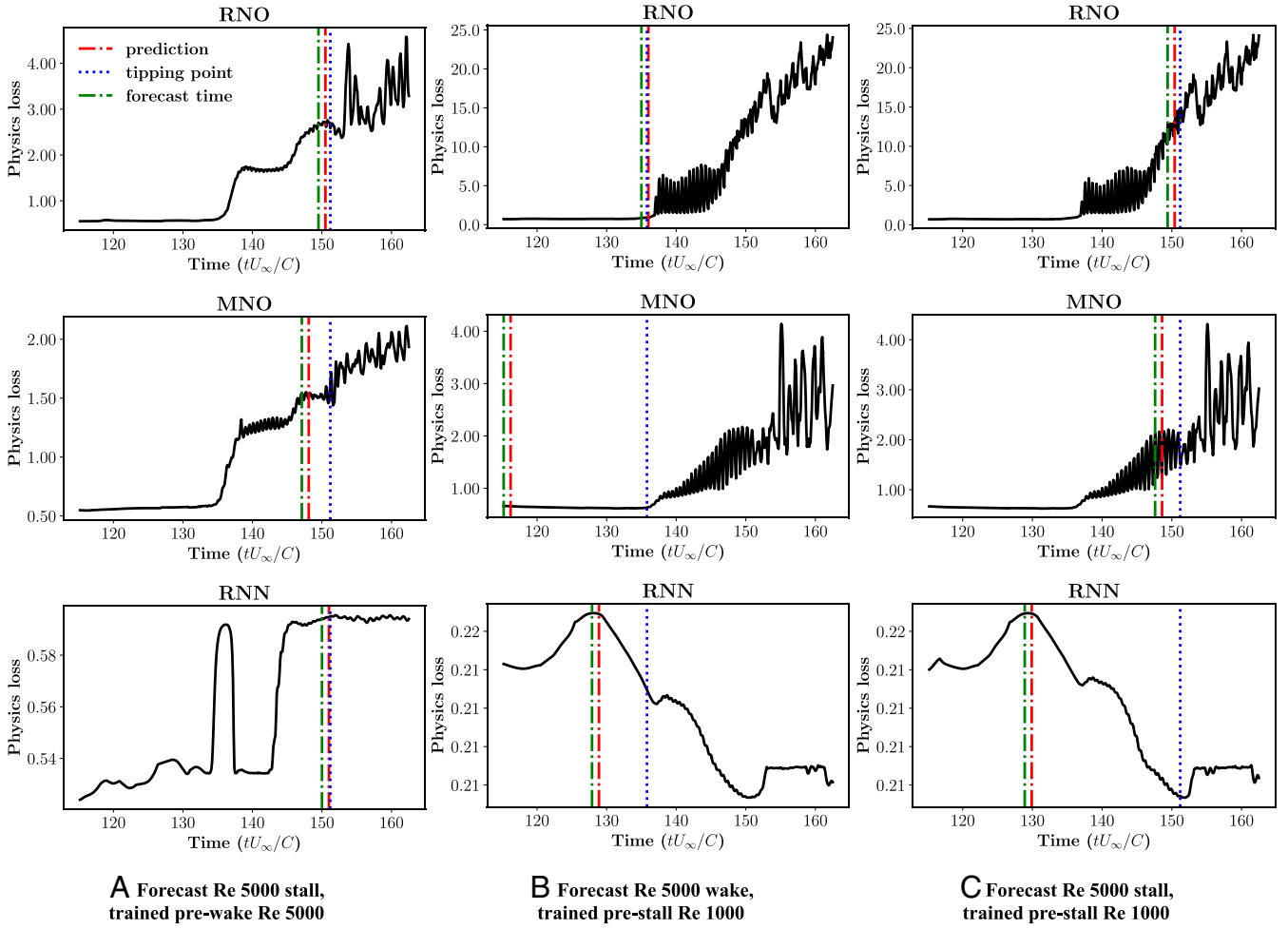


Fig. 6. Tipping point forecast comparison between RNO, MNO, and RNN in three generalization settings for the flow over the airfoil setting. (*A*) considers forecasting the stall transition in a Re 5000 flow with the dynamics model trained on pre-wake Re 5000 data. (*B*) considers forecasting the wake transition in a Re 5000 flow with the dynamics model trained on pre-stall Re 1000 data. (*C*) considers forecasting the stall transition in a Re 5000 flow with the dynamics model trained on pre-stall Re 1000 data. (*A* and *C*) use $\alpha = 0.11$, and (*B*) uses $\alpha = 0.14$.

note that this physical constraint assumes only minimal physical knowledge of the system.

In our experiments, we also compare the tipping point forecasting performance using RNO, MNO, and RNN. The full results for these settings can be found in *SI Appendix*, section S6, and experiments exploring robustness to different rates of change of the airfoil angle of attack, as well as nontipping settings, can be found in *SI Appendix*, section S7. We find that RNO produces the most accurate forecasts for each setting, over various choices of α. However, MNO and RNN are often still able to forecast the transitions, suggesting that our method is robust to the quality of the temporal evolution model.

These results motivate the evaluation of the generalizability of our methodology to future tipping points. Fig. 6*A* shows the results of the RNO, MNO, and RNN trained on pre-wake data for Re 5000 but used to forecast the stall tipping point for Re 5000 at inference time. Once again, we observe that the stall point is mostly correctly forecasted by all three models.

Last, we evaluate the performance of our methods on generalizing to forecasting tipping points at different Reynolds numbers. Specifically, we train each of the three models on Re 1000 pre-stall data and use these models to forecast the wake and stall tipping points in the Re 5000 setting. Fig. 6*B* shows the results of RNO, MNO, and RNN trained on Re 1000 pre-stall dynamics and used directly to forecast the Re 5000 wake tipping point. Fig. 6*C* shows the analogous results for the Re 5000 stall tipping point. The results show that RNO far outperforms MNO and RNN in forecasting the tipping points. This suggests that the generalization ability of our tipping point forecasting method depends on the accuracy of the underlying dynamics model.

4.4. Discussion. From the experiments above (and those in the appendix), we can extract several key takeaways about our proposed tipping point forecasting method:

1. Tipping point forecasting remains robust under approximate physical constraints (*SI Appendix*, section S1.C).
2. Tipping point forecasting remains robust using a physics loss based on partial physics laws (i.e., not the full governing equations) and conservation laws (Sections 4.2 and 4.3).
3. Our proposed method is easily applicable to large-scale systems with complex tipping points, even when knowledge of the full governing equations is not known (Section 4.3).
4. Our proposed method generalizes to forecasting tipping points occurring later in time and across different regimes than seen during training (e.g., different Reynolds numbers for the airfoil experiment), although the robustness depends on the generalizability of the models used to learn the nonstationary dynamics (Section 4.3).

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

Data, Materials, and Software Availability. Code and training dataset have been deposited in Github (https://github.com/neuraloperator/tipping-point-forecast/) (70) and Zenodo (https://zenodo.org/records/18098807) (71). All other data are included in the manuscript and/or *SI Appendix*.

ACKNOWLEDGMENTS. We thank Tapio Schneider for the helpful discussion. M.L.-S. was supported in part by the Mellon Mays Undergraduate Fellowship. C.E.S. was supported by the generosity of Eric and Wendy Schmidt by recommendation of the Schmidt Futures program and by Charles Trimble. A.A. is supported in part by Bren endowed chair. We gratefully acknowledge the funding provided for the airfoil research by the SciAI Center, supported by the Office of Naval Research (ONR) under grant number N00014-23-1-2729, the Carver Mead New Adventures Fund, and the Center for Autonomous Systems and Technologies at the California Institute of Technology. Computational time for running the airfoil stall simulation was provided by the Discover project at Pittsburgh Supercomputing Center through allocation PHY240020 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by NSF grants No. 2138259, No. 2138286, No. 2138307, No. 2137603, and No. 2138296. We gratefully acknowledge the funding provided for this research by the Keck Scholar-Fellow Bridge Initiative from the W. M. Keck Foundation, the SciAI Center, supported by the ONR under grant number N00014-23-1-2729, the Carver Mead New Adventures Fund, and the Center for Autonomous Systems and Technologies at the California Institute of Technology. Computational time was provided by the Discover project at Pittsburgh Supercomputing Center through allocation PHY240020 from the ACCESS program, which is supported by NSF grants No. 2138259, No. 2138286, No. 2138307, No. 2137603, and No. 2138296.

1. D. Lemoine, C. Traeger, Watch your step: Optimal policy in a tipping climate. *Am. Econ. J. Econ. Policy* **6**, 137–166 (2014).
2. T. Schneider, C. M. Kaul, K. G. Pressel, Possible climate transitions from breakup of stratocumulus decks under greenhouse warming. *Nat. Geosci.* **12**, 163–167 (2019).
3. C. E. Singer, T. Schneider, Stratocumulus-cumulus transition explained by bulk boundary layer theory. *ESS Open Arch.* **2023**, 1–30 (2023).
4. C. E. Singer, T. Schneider, Co 2 -driven stratocumulus cloud breakup in a bulk boundary layer model. *ESS Open Arch.*, 10.22541/essoar.168167204.45220772/v1 (2023).
5. R. S. Rogallo, P. Moin, Numerical simulation of turbulent flows. *Annu. Rev. Fluid Mech.* **16**, 99–137 (1984).
6. D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning internal representations by error propagation" in *Parallel distributed processing: Explorations in the microstructures of cognition, Vol. I*, D. E. Rumelhart, J. L. McClelland Eds. (Cambridge, MIT Press, MA, 1988), pp. 318–362.
7. D. Patel, E. Ott, Using machine learning to anticipate tipping points and extrapolate to post-tipping dynamics of non-stationary dynamical systems. *Chaos* **33**, 023143 (2023).
8. N. Kovachki *et al*., Neural operator: Learning maps between function spaces with applications to pdes. *J. Mach. Learn. Res.* **24**, 1–97 (2023).
9. K. Azizzadenesheli *et al*., Neural operators for accelerating scientific simulations and design. *Nat. Rev. Phys.* **6**, 320–328 (2024).
10. J. Berner *et al*., Principled approaches for extending neural architectures to function spaces for operator learning. *Nat Mach Intell* **8**, 1173–1181 (2026).
11. Z. Li *et al*., Learning chaotic dynamics in dissipative systems. *Adv. Neural Inf. Process. Syst.* **35**, 16768–16781 (2022).
12. P. Ashwin, S. Wieczorek, R. Vitolo, P. Cox, Tipping points in open systems: Bifurcation, noise-induced and rate-dependent examples in the climate system. *Philos. Trans. R. Soc. A Math. Phys. Eng. Sci.* **370**, 1166–1184 (2012).
13. T. M. Lenton *et al*., Tipping elements in the earth's climate system. *Proc. Natl. Acad. Sci. U.S.A.* **105**, 1786–1793 (2008).
14. D. I. Armstrong McKay *et al*., Exceeding 1.5°C global warming could trigger multiple climate tipping points. *Science* **377**, eabn7950 (2022).
15. S. Wang *et al*., Mechanisms and impacts of earth system tipping elements. *Rev. Geophys.* **61**, e2021RG000757 (2023).
16. S. Rahmstorf, Thermohaline ocean circulation. *Encycl. Quat. Sci.* **5**, 1–10 (2006).
17. D. B. Bonan, A. F. Thompson, E. R. Newsom, S. Sun, M. Rugenstein, Transient and equilibrium responses of the Atlantic overturning circulation to warming in coupled climate models: The role of temperature and salinity. *J. Clim.* **35**, 5173–5193 (2022).
18. D. Rind *et al*., Multicentury instability of the Atlantic meridional circulation in rapid warming simulations with giss modele2. *J. Geophys. Res. Atmos.* **123**, 6331–6355 (2018).
19. M. Mengel, A. Levermann, Ice plug prevents irreversible discharge from east Antarctica. *Nat. Clim. Change* **4**, 451–455 (2014).
20. L. Favier *et al*., Retreat of pine island glacier controlled by marine ice-sheet instability. *Nat. Clim. Change* **4**, 117–121 (2014).
21. H. Seroussi *et al*., Sensitivity of the dynamics of pine island glacier, west Antarctica, to climate forcing for the next 50 years. *Cryosphere* **8**, 1699–1710 (2014).
22. I. Joughin, D. Shapero, P. Dutrieux, B. Smith, Ocean-induced melt volume directly paces ice loss from pine island glacier. *Sci. Adv.* **7**, eabi5738 (2021).
23. C. D. Koven *et al*., A simplified, data-constrained approach to estimate the permafrost carbon-climate feedback. *Philos. Trans. R. Soc. A Math. Phys. Eng. Sci.* **373**, 20140423 (2015).
24. K. R. Miner *et al*., Permafrost carbon emissions in a changing arctic. *Nat. Rev. Earth Environ.* **3**, 55–67 (2022).
25. S. Gupta *et al*., Two-and three-dimensional wake transitions of a naca0012 airfoil. *J. Fluid Mech.* **954**, A26 (2023).
26. W. J. McCroskey, "The phenomenon of dynamic stall" (NASA Technical Memorandum NASA-TM-81264, NASA, Ames Research Center, Moffett Field, California, 1981).
27. T. M. Bury *et al*., Deep learning for early warning signals of tipping points. *Proc. Natl. Acad. Sci. U.S.A.* **118**, e2106140118 (2021).
28. D. Patel, D. Canaday, M. Girvan, A. Pomerance, E. Ott, Using machine learning to predict statistical properties of non-stationary dynamical processes: System climate, regime transitions, and the effect of stochasticity. *Chaos Interdiscip. J. Nonlinear Sci.* **31**, 033149 (2021).
29. L. W. Kong, H. W. Fan, C. Grebogi, Y. C. Lai, Machine learning prediction of critical transition and system collapse. *Phys. Rev. Res.* **3**, 013090 (2021).
30. S. H. Lim, L. Theo Giorgini, W. Moon, J. S. Wettlaufer, Predicting critical transitions in multiscale dynamical systems using reservoir computing. *Chaos Interdiscip. J. Nonlinear Sci.* **30**, 123126 (2020).
31. X. Li *et al*., Tipping point detection using reservoir computing. *Research* **6**, 0174 (2023).
32. S. Deb, S. Sidheekh, C. F. Clements, N. C. Krishnan, P. S. Dutta, Machine learning methods trained on simple models can predict critical transitions in complex natural systems. *R. Soc. Open Sci.* **9**, 211475 (2022).
33. J. Sleeman *et al*., A generative adversarial network for climate tipping point discovery (tip-gan). arXiv [Preprint] (2023). https://arxiv.org/abs/2302.10274.
34. Y. Huang, S. Bathiany, P. Ashwin, N. Boers, Deep learning for predicting rate-induced tipping. *Nat. Mach. Intell.* **6**, 1–10 (2024).
35. S. Panahi *et al*., Machine learning prediction of tipping in complex dynamical systems. *Phys. Rev. Res.* **6**, 043194 (2024).
36. G. Fabiani *et al*., Task-oriented machine learning surrogates for tipping points of agent-based models. *Nat. Commun.* **15**, 4117 (2024).
37. S. Panahi, L. W. Kong, B. Glaz, M. Haile, Y. C. Lai, Unsupervised learning for anticipating critical transitions. *Phys. Rev. Lett.* **136**, 077301 (2026).
38. H. Hersbach *et al*., The era5 global reanalysis. *Q. J. R. Meteorol. Soc.* **146**, 1999–2049 (2020).
39. G. Shafer, V. Vovk, A tutorial on conformal prediction. *J. Mach. Learn. Res.* **9**, 1–51 (2008).
40. A. Dvoretzky, J. Kiefer, J. Wolfowitz, Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *Ann. Math. Stat.* **27**, 642–669 (1956).
41. P. Massart, The tight constant in the Dvoretzky-Kiefer-Wolfowitz inequality. *Ann. Probab.* **18**, 1269–1283 (1990).
42. P. Ashwin, J. Newman, Physical invariant measures and tipping probabilities for chaotic attractors of asymptotically autonomous systems. *Eur. Phys. J. Special Top.* **230**, 3235–3248 (2021).
43. B. Kaszás, U. Feudel, T. Tél, Tipping phenomena in typical dynamical systems subjected to parameter drift. *Sci. Rep.* **9**, 1–12 (2019).
44. S. Panahi, Y. C. Lai, Global phase-space approach to rate-induced tipping: A brief review. *Chaos Interdiscip. J. Nonlinear Sci.* **35**, 043139 (2025).
45. M. Scheffer *et al*., Early-warning signals for critical transitions. *Nature* **461**, 53–59 (2009).
46. T. M. Lenton, Early warning of climate tipping points. *Nat. Clim. Change* **1**, 201–209 (2011).
47. M. Scheffer *et al*., Anticipating critical transitions. *Science* **338**, 344–348 (2012).
48. P. D. Ditlevsen, S. J. Johnsen, Tipping points: Early warning and wishful thinking. *Geophys. Res. Lett.* **37**, e2010GL044486 (2010).
49. V. Dakos *et al*., Tipping point detection and early warnings in climate, ecological, and human systems. *Earth Syst. Dyn.* **15**, 1117–1135 (2024).
50. V. Lucarini, M. D. Chekroun, Theoretical tools for understanding the climate crisis from Hasselmann's programme and beyond. *Nat. Rev. Phys.* **5**, 744–765 (2023).
51. V. Lucarini, M. D. Chekroun, Detecting and attributing change in climate and complex systems: Foundations, green's functions, and nonlinear fingerprints. *Phys. Rev. Lett.* **133**, 244201 (2024).
52. A. Tantet, V. Lucarini, H. A. Dijkstra, Resonances in a chaotic attractor crisis of the lorenz flow. *J. Stat. Phys.* **170**, 584–616 (2018).
53. V. Gopakumar, S. Pamela, L. Zanisi, Fourier-rnns for modelling noisy physics data. arXiv [Preprint] (2023). https://arxiv.org/abs/2302.06534.
54. Y. Rubanova, R. T. Chen, D. K. Duvenaud, Latent ordinary differential equations for irregularly-sampled time series. *Adv. Neural Inf. Process. Syst.* **32** (2019).
55. M. Habiba, B. A. Pearlmutter, "Neural ordinary differential equation based recurrent neural network model" in *2020 31st Irish signals and systems conference (ISSC)* (IEEE, 2020), pp. 1–6.
56. B. Liu, M. Trautner, A. M. Stuart, K. Bhattacharya, Learning macroscopic internal variables and history dependence from microscopic models. *J. Mech. Phys. Solids.* **178**, 105329 (2023).
57. Z. Li *et al*., "Fourier neural operator for parametric partial differential equations" in *International Conference on Learning Representations* (2021). https://openreview.net/forum?id=c8P9NQVtmnO. Accessed 1 August 2023.
58. M. A. Rahman, Z. E. Ross, K. Azizzadenesheli, U-no: U-shaped neural operators. arXiv [Preprint] (2022). https://arxiv.org/abs/2204.11127.
59. M. Liu-Schiaffini *et al*., Tipping point forecasting in non-stationary dynamics on function spaces. arXiv [Preprint] (2023). https://arxiv.org/abs/2308.08794.
60. R. Buitrago, T. Marwah, A. Gu, A. Risteski, "On the benefits of memory for modeling time-dependent PDEs" in *The Thirteenth International Conference on Learning Representations* (2025). https://openreview.net/forum?id=o9kqa5K3tB. Accessed 31 December 2025.
61. B. Guo, Z. Lin, Q. He, History-aware neural operator: Robust data-driven constitutive modeling of path-dependent materials. arXiv [Preprint] (2025). https://arxiv.org/abs/2506.10352 (Accessed 31 December 2025).
62. Z. Ye, C. S. Zhang, W. Wang, Recurrent neural operators: Stable long-term pde prediction. arXiv [Preprint] (2025). https://arxiv.org/abs/2505.20721.
63. N. F. Koren, S. Lanthaler, Merging memory and space: A spatiotemporal state space neural operator. arXiv [Preprint] (2025). https://arxiv.org/abs/2507.23428 (Accessed 31 December 2025).

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.

64. K. Cho *et al*., Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv [Preprint] (2014). https://arxiv.org/abs/1406.1078 (Accessed 1 August 2025).
65. I. Takeuchi *et al*., Nonparametric quantile estimation. *J. Mach. Learn. Res.* **7**, 1231–1264 (2006).
66. A. Huang, L. Leqi, Z. Lipton, K. Azizzadenesheli, Off-policy risk assessment in contextual bandits. *Adv. Neural Inf. Process. Syst.* **34**, 23714–23726 (2021).
67. T. Schneider, C. M. Kaul, K. G. Pressel, Solar geoengineering may not prevent strong warming from direct effects of CO_2 on stratocumulus cloud cover. *Proc. Natl. Acad. Sci. U.S.A.* **117**, 30179–30185 (2020).
68. S. C. Leung, D. Zhou, H. J. Bae, Smart sensor placement: A correlation-aware attribution framework (caaf). for real-world data modeling. arXiv [Preprint] (2025). https://arxiv.org/abs/2510.22517.
69. D. F. Kurtulus, On the unsteady behavior of the flow around NACA 0012 airfoil with steady external conditions at Re = 1000. *Int. J. Micro Air Veh.* **7**, 301–326 (2015).
70. M. Liu-Schiaffini, tipping-point-forecast [Source code]. GitHub. https://github.com/neuraloperator/tipping-point-forecast/. Deposited 1 January 2026
71. M. Liu-Schiaffini *et al*., Tipping point forecasting in non-stationary dynamics on function spaces (airfoil dataset) (Version 1.0.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.18098807. Deposited 30 December 2025.

Downloaded from https://www.pnas.org by 219.77.125.56 on November 11, 2026 from IP address 219.77.125.56.